LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – RELASI ANTAR KELAS



Keyla Azzahra

103132400017

S1SD-05

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Object Oriented Programming (OOP) merupakan pendekatan pemrograman yang menggunakan kelas dan objek untuk menyusun program. Dalam OOP, kelas digunakan sebagai rancangan untuk membuat objek yang memiliki atribut dan method. Atribut digunakan untuk menyimpan data, sedangkan method digunakan untuk menjalankan operasi tertentu. Diagram kelas merupakan diagram yang digunakan untuk memodelkan kelas-kelas dalam suatu sistem beserta hubungan antar kelas tersebut. Diagram kelas terdiri dari nama kelas, atribut, dan method. Simbol + menunjukkan akses public, simbol - menunjukkan akses private, sedangkan simbol # menunjukkan akses protected. Dalam pemrograman berorientasi objek, terdapat beberapa jenis relasi antar kelas, yaitu asosiasi, agregasi, dan komposisi. Dalam pemrograman berorientasi objek, terdapat beberapa jenis relasi antar kelas, yaitu asosiasi, agregasi, dan komposisi.

1. Asosiasi (Association)

   Asosiasi merupakan hubungan antara dua kelas yang berbeda dan dapat berinteraksi melalui objek masing-masing. Pada hubungan ini, suatu kelas dapat menggunakan method dari kelas lain tanpa harus menyimpan objek tersebut sebagai atribut permanen. Contohnya adalah hubungan antara kelas Kasir dan Pesanan, ketika kasir memproses pembayaran dengan menerima objek pesanan sebagai parameter method.
2. Agregasi (Aggregation)

   Agregasi merupakan hubungan antara bagian dan keseluruhan (part-of), tetapi objek bagian tetap dapat dibuat dan berdiri sendiri tanpa bergantung pada objek keseluruhan. Contohnya adalah kelas Pesanan yang menampung beberapa objek ItemPesanan menggunakan array. Objek item dibuat terlebih dahulu, kemudian dimasukkan ke dalam pesanan.
3. Komposisi (Composition)

   Komposisi merupakan bentuk hubungan bagian dan keseluruhan yang lebih kuat dibandingkan agregasi. Objek bagian memiliki siklus hidup yang terikat pada objek utamanya. Contohnya adalah kelas ItemPesanan yang memiliki objek RincianHarga. Objek RincianHarga dibuat langsung di dalam constructor ItemPesanan.

Selain itu, penerapan enkapsulasi diperlukan untuk melindungi data di dalam kelas. Enkapsulasi dilakukan dengan menggunakan atribut private dan menyediakan method untuk mengakses data tersebut. Dengan demikian, data objek tidak diakses atau diubah secara langsung dari luar kelas.

B. Guided

1. Asosiasi

Mahasiswa.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java
to edit this template
 */
package guided;

/**
 *
 * @author user
 */
public class Mahasiswa {

    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNilai(int nilai){
        this.nilai = nilai;
    }

    //getter
    public int getNilai(){
        return this.nilai;
    }

    public String getNama() {
        return this.nama;
    }

    public String getNim() {
        return this.nim;
    }
```

```
    }
```

Dosen.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided;

/**
 *
 * @author user
 */
public class Dosen {

    private String kodeDosen;
    private String namaDosen;
//Setter

    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }
//Getter

    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
        // ini asosiasi, method milik class Student dipanggil di class
// tp objek Student tidak menjadi atribut dr class Lecture
s.setNilai(nilai);
    }
```

```
    public int getScore(Mahasiswa s) {
        // ini asosiasi, method milik class Student dipanggil di class
// tp objek Student tidak menjadi atribut dr class Lecture
return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("103132400017");
        m.setNama("Key");
        Dosen d = new Dosen();
        d.giveScore(m, 100);
        System.out.println("NIM :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul5 ---
NIM :103132400017
Nama :Key
Nilai :100
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. Agregasi

Mahasiswa.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package guided2;

/**
 *
 * @author user
 */
public class Mahasiswa {
```

```
    private String NIM;
    private String Nama;

    public Mahasiswa(String no, String nm) {
        this.NIM = no;
        this.Nama = nm;
    }

    public String GetNIM() {
        return this.NIM;
    }

    public String GetNama() {
        return this.Nama;
    }
}
```

Jurusan.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided2;

/**
 *
 * @author user
 */
public class Jurusan {
    private String KodeJurusan;
    private String NamaJurusan;
    private Mahasiswa[] Daftar = new Mahasiswa[10];
    private int JmlMhs = 0;

    public Jurusan(String kode, String nama) {
        this.KodeJurusan = kode;
        this.NamaJurusan = nama;
    }

    public void AddMahasiswa(Mahasiswa m) {
        if (this.JmlMhs < this.Daftar.length) {
            this.Daftar[this.JmlMhs] = m;
            this.JmlMhs++;
```

```
        }
    }

    public void DisplayMahasiswa() {
        System.out.println("Kode Jurusan : " + this.KodeJurusan);
        System.out.println("Nama Jurusan : " + this.NamaJurusan);
        System.out.println("Daftar Mahasiswa :");

        for (int i = 0; i < this.JmlMhs; i++) {
            System.out.println(
                this.Daftar[i].GetNIM() + " "
                + this.Daftar[i].GetNama()
            );
        }
    }
}
```

Main.java

```
package guided2;

public class Main {
    public static void main(String[] args) {
        Mahasiswa m1 = new Mahasiswa("101", "Andi");
        Mahasiswa m2 = new Mahasiswa("102", "Siti");

        Jurusan j = new Jurusan("SD01", "Sains Data");

        j.AddMahasiswa(m1);
        j.AddMahasiswa(m2);

        j.DisplayMahasiswa();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul5 ---
Kode Jurusan : SD01
Nama Jurusan : Sains Data
Daftar Mahasiswa :
103132400017 Keyla Azzahra
103132430021 Raymundus Ariel Abas
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

3. Komposisi

Komputer.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Komposisi;

/**
 *
 * @author user
 */
public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }

}
```

Monitor.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package Komposisi;

/**
 *
 * @author user
 */
public class Monitor {
    private String brand;

   public Monitor(String brand){
      this.brand = brand;
   }

   public void DisplaySpec() {
      System.out.println(this.brand);
   }
}
```

Mouse.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package Komposisi;

/**
 *
 * @author user
 */
public class Mouse {
      private String brand, tipe;

   public Mouse(String brand, String tipe){
      this.brand = brand;
      this.tipe = tipe;
```

```
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

Processor.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package Komposisi;

/**
 *
 * @author user
 */
public class Processor {
     private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul5 ---
Nama: Keyla Azzahra
NIM: 103132400017
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 1600
Lenovo ThinkPad T490
Lenovo, Wireless
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

Buatlah dua class, yaitu RincianHarga dan ItemPesanan. Setiap item pesanan memiliki rincian harga yang mencatat harga dasar dan pajak sebesar 10%. Objek RincianHarga harus dibuat langsung di dalam constructor ItemPesanan dan tidak dibuat di dalam Main.

RincianHarga.java

Jawaban :

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package Unguided;

/**
 *
 * @author user
 */
public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }
```

```
    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

ItemPesanan.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package Unguided;

/**
 *
 * @author user
 */
public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

Output :
Output pengujian kedua class ini ditampilkan melalui Main pada bagian Unguided 3.

Penjelasan :
Class RincianHarga memiliki atribut hargaDasar dan pajak yang bertipe double. Constructor digunakan untuk mengisi harga dasar dan menghitung pajak sebesar 10% dari harga dasar. Method getHargaTotal() digunakan untuk menjumlahkan harga dasar dengan pajak. Class ItemPesanan memiliki atribut namaItem dan rincian. Pada constructor ItemPesanan, objek RincianHarga dibuat menggunakan kata kunci new. Hal tersebut menunjukkan penerapan komposisi karena objek rincian harga dibuat sebagai bagian internal dari item pesanan. Atribut dibuat private agar penerapan enkapsulasi tetap terjaga.

2. Soal Unguided 2
Buatlah class Pesanan yang dapat menampung beberapa objek ItemPesanan menggunakan array standar. Objek ItemPesanan dibuat terlebih dahulu secara terpisah, kemudian dimasukkan ke dalam objek Pesanan menggunakan method tambahItem().

Pesanan.java
Jawaban :

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package Unguided;

/**
 *
 * @author user
 */
public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[5];
        this.jumlahItem = 0;
    }
```

```
    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        } else {
            System.out.println("Pesanan sudah penuh!");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);

        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            RincianHarga rincian = item.getRincian();

            System.out.println(
                (i + 1) + ". " + item.getNamaItem()
                + " - Rp " + rincian.getHargaTotal()
                + " (Harga: " + rincian.getHargaDasar()
                + ", Pajak: " + rincian.getPajak() + ")"
            );
        }
    }

    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
```

Output :

Output daftar pesanan akan ditampilkan saat method tampilkanPesanan() dipanggil pada bagian Unguided 3.

Penjelasan :

Class Pesanan memiliki tiga atribut, yaitu nomorMeja, daftarItem, dan jumlahItem. Atribut daftarItem merupakan array bertipe ItemPesanan yang memiliki kapasitas lima objek.
Constructor Pesanan digunakan untuk menentukan nomor meja, membuat array, dan menginisialisasi jumlah item menjadi nol. Method tambahItem() digunakan untuk memasukkan objek ke dalam array selama kapasitas masih tersedia.
Method tampilkanPesanan() menampilkan nomor meja dan rincian seluruh item, termasuk harga dasar, pajak, dan harga total. Method getter digunakan untuk memperoleh data yang diperlukan oleh kelas lain.
Hubungan agregasi terlihat ketika objek ItemPesanan dibuat secara terpisah di Main, kemudian dimasukkan ke dalam objek Pesanan. Dengan demikian, objek item tidak harus dibuat langsung oleh kelas Pesanan.

3. Soal Unguided 3
Buatlah class Kasir yang dapat memproses pembayaran dari objek Pesanan. Class Kasir tidak boleh menyimpan objek Pesanan sebagai atribut. Selanjutnya, buat class Main untuk menguji program dengan satu kasir, tiga item pesanan, dan satu pesanan untuk meja nomor 4.

Kasir.java
Jawaban :

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package Unguided;

/**
 *
 * @author user
 */
public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;
```

```
        ItemPesanan[] daftarItem = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga rincian =
                daftarItem[i].getRincian();

            totalSubtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println();
        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja "
                + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : "
                + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

Main.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided2;
/**
 *
 * @author user
 */
public class Main {
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");

        ItemPesanan item1 =
```

```
                    new ItemPesanan("Iced Americano", 18000);

                ItemPesanan item2 =
                    new ItemPesanan("Beef Croissant", 25000);

                ItemPesanan item3 =
                    new ItemPesanan("Matcha Latte", 22000);

                Pesanan pesanan = new Pesanan(4);

                pesanan.tambahItem(item1);
                pesanan.tambahItem(item2);
                pesanan.tambahItem(item3);

                pesanan.tampilkanPesanan();
                kasir.prosesPembayaran(pesanan);
            }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Class Kasir memiliki atribut namaKasir untuk menyimpan nama petugas. Method prosesPembayaran() menerima objek Pesanan sebagai parameter, kemudian mengambil daftar item dan menghitung subtotal serta total pajak menggunakan perulangan. Subtotal diperoleh dari penjumlahan seluruh harga dasar item, sedangkan total pajak diperoleh dari penjumlahan pajak setiap item. Total pembayaran merupakan hasil penjumlahan subtotal dan total pajak. Class Main berfungsi sebagai kelas utama

untuk menjalankan program. Pertama, objek Kasir dibuat dengan nama Budi. Selanjutnya, tiga objek ItemPesanan dibuat secara independen dengan nama menu dan harga dasar masing-masing. Setelah itu, objek Pesanan dibuat untuk meja nomor 4 dan ketiga item dimasukkan menggunakan method tambahItem().

Method tampilkanPesanan() digunakan untuk menampilkan daftar menu, sedangkan prosesPembayaran() digunakan untuk menghasilkan struk pembayaran. Hubungan asosiasi ditunjukkan ketika objek Kasir menggunakan objek Pesanan melalui parameter method tanpa menyimpan objek tersebut sebagai atribut kelas.

D. Kesimpulan

Berdasarkan praktikum Pemrograman Berorientasi Objek Modul 5, dapat disimpulkan bahwa relasi antar kelas terdiri dari asosiasi, agregasi, dan komposisi. Ketiga relasi tersebut memiliki karakteristik yang berbeda sesuai dengan hubungan antarobjek di dalam program. Komposisi diterapkan pada class ItemPesanan dan RincianHarga, yaitu ketika objek rincian harga dibuat di dalam constructor item pesanan. Agregasi diterapkan pada class Pesanan dan ItemPesanan, yaitu ketika objek item dibuat secara terpisah lalu dimasukkan ke dalam array pesanan. Sementara itu, asosiasi diterapkan pada class Kasir dan Pesanan, yaitu ketika kasir memproses pembayaran melalui parameter method. Praktikum ini juga menerapkan enkapsulasi dengan menggunakan atribut private, constructor, dan method getter untuk mengakses data. Program yang dibuat dapat menampilkan daftar pesanan serta menghitung subtotal, pajak, dan total pembayaran. Dengan demikian, praktikum ini membantu memahami penerapan relasi antar kelas dalam pemrograman Java sesuai dengan konsep Object Oriented Programming.